1. Implement A* Search for 8 puzzle problem

```
import heapq

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def manhattan_distance(state):
    distance = 0
    for i, tile in enumerate(state):
        if tile == 0:
            continue
        goal_index = tile - 1
        curr_row, curr_col = i // 3, i % 3
        goal_row, goal_col = goal_index // 3, goal_index % 3
        distance += abs(curr_row - goal_row) + abs(curr_col - goal_col)
    return distance

def get_neighbors(state):
    neighbors = []
    state = list(state)
    blank = state.index(0)
    row, col = blank // 3, blank % 3
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    for dr, dc in moves:
        new_row, new_col = row + dr, col + dc
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_blank = new_row * 3 + new_col
            new_state = state[:]
            new_state[blank], new_state[new_blank] = new_state[new_blank], new_state[blank]
            neighbors.append(tuple(new_state))
    return neighbors

def a_star(start):
    start = tuple(start)
    h = manhattan_distance(start)
    heap = [(h, 0, start, [])]
    visited = set()
    while heap:
        f, g, state, path = heapq.heappop(heap)
        if state in visited:
            continue
        visited.add(state)
        if state == GOAL:
            return path + [state]
        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                new_g = g + 1
                new_h = manhattan_distance(neighbor)
                new_f = new_g + new_h
                heapq.heappush(heap, (new_f, new_g, neighbor, path + [state]))
    return None

def print_board(state):
    for i in range(0, 9, 3):
        row = state[i:i+3]
        print(" ".join(str(x) if x != 0 else "_" for x in row))
    print()

if __name__ == "__main__":
    start = (1, 2, 3, 4, 0, 6, 7, 5, 8)
    print("Start State:")
    print_board(start)
    solution = a_star(start)
    if solution:
        print(f"Solution found in {len(solution) - 1} moves!\n")
        for step, state in enumerate(solution):
            print(f"Step {step}:")
            print_board(state)
    else:
        print("No solution exists.")
```

2. Implement Tower of Hanoi by State Space Search

```
from collections import deque

def get_initial_state(n):
    return (tuple(range(n, 0, -1)), (), ())

def get_goal_state(n):
    return ((), (), tuple(range(n, 0, -1)))

def get_neighbors(state):
    neighbors = []
    pegs = list(state)
    for i in range(3):
        if not pegs[i]:
            continue
        top = pegs[i][-1]
        for j in range(3):
            if i == j:
                continue
            if not pegs[j] or pegs[j][-1] > top:
                new_pegs = [list(p) for p in pegs]
                new_pegs[i].pop()
                new_pegs[j].append(top)
                neighbors.append((tuple(new_pegs[0]), tuple(new_pegs[1]), tuple(new_pegs[2])))
    return neighbors

def bfs(n):
    start = get_initial_state(n)
    goal = get_goal_state(n)
    queue = deque()
    queue.append((start, [start]))
    visited = set()
    visited.add(start)
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path
        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))
    return None

def print_state(state, step):
    print(f"Step {step}:")
    labels = ['A', 'B', 'C']
    for i, peg in enumerate(state):
        print(f"  Peg {labels[i]}: {list(peg)}")
    print()

if __name__ == "__main__":
    n = 3
    print(f"Tower of Hanoi with {n} disks\n")
    solution = bfs(n)
    if solution:
        print(f"Solution found in {len(solution) - 1} moves!\n")
        for step, state in enumerate(solution):
            print_state(state, step)
    else:
        print("No solution found.")
```

3.Use BFS to solve a planning problem. Examples can be 8 puzzle problem.

```
from collections import deque

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def get_neighbors(state):
    neighbors = []
    state = list(state)
    blank = state.index(0)
    row, col = blank // 3, blank % 3
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    for dr, dc in moves:
        new_row, new_col = row + dr, col + dc
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_blank = new_row * 3 + new_col
            new_state = state[:]
            new_state[blank], new_state[new_blank] = new_state[new_blank], new_state[blank]
            neighbors.append(tuple(new_state))
    return neighbors

def bfs(start):
    start = tuple(start)
    if start == GOAL:
        return [start]
    queue = deque()
    queue.append((start, [start]))
    visited = set()
    visited.add(start)
    while queue:
        state, path = queue.popleft()
        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                if neighbor == GOAL:
                    return path + [neighbor]
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))
    return None

def print_board(state):
    for i in range(0, 9, 3):
        row = state[i:i+3]
        print(" ".join(str(x) if x != 0 else "_" for x in row))
    print()

if __name__ == "__main__":
    start = (1, 2, 3, 4, 0, 6, 7, 5, 8)
    print("Start State:")
    print_board(start)
    solution = bfs(start)
    if solution:
        print(f"Solution found in {len(solution) - 1} moves!\n")
        for step, state in enumerate(solution):
            print(f"Step {step}:")
            print_board(state)
    else:
        print("No solution exists.")
```

4. Use DFS for problem solving. Examples can be water jug problem

```
def get_neighbors(state, jug1_cap, jug2_cap):
    j1, j2 = state
    neighbors = []

    neighbors.append((jug1_cap, j2))
    neighbors.append((j1, jug2_cap))

    neighbors.append((0, j2))
    neighbors.append((j1, 0))

    pour = min(j1, jug2_cap - j2)
    neighbors.append((j1 - pour, j2 + pour))

    pour = min(j2, jug1_cap - j1)
    neighbors.append((j1 + pour, j2 - pour))

    return neighbors

def dfs(jug1_cap, jug2_cap, target):
    start = (0, 0)
    stack = [(start, [start])]
    visited = set()
    visited.add(start)

    while stack:
        state, path = stack.pop()
        j1, j2 = state

        if j1 == target or j2 == target:
            return path

        for neighbor in get_neighbors(state, jug1_cap, jug2_cap):
            if neighbor not in visited:
                visited.add(neighbor)
                stack.append((neighbor, path + [neighbor]))

    return None

def print_state(state, step):
    print(f"Step {step}: Jug1 = {state[0]}L, Jug2 = {state[1]}L")

if __name__ == "__main__":
    jug1_cap = 4
    jug2_cap = 3
    target = 2

    print(f"Jug1 Capacity: {jug1_cap}L")
    print(f"Jug2 Capacity: {jug2_cap}L")
    print(f"Target: {target}L\n")

    solution = dfs(jug1_cap, jug2_cap, target)

    if solution:
        print(f"Solution found in {len(solution) - 1} steps!\n")
        for step, state in enumerate(solution):
            print_state(state, step)
    else:
        print("No solution exists.")
```

5. Implement Adversarial Search with Alpha-Beta pruning

```
import math

def print_board(board):
    for row in board:
        print(" | ".join(row))
        print("-" * 9)
    print()

def check_winner(board):
    for row in board:
        if row[0] == row[1] == row[2] and row[0] != " ":
            return row[0]
    for col in range(3):
        if board[0][col] == board[1][col] == board[2][col] and board[0][col] != " ":
            return board[0][col]
    if board[0][0] == board[1][1] == board[2][2] and board[0][0] != " ":
        return board[0][0]
    if board[0][2] == board[1][1] == board[2][0] and board[0][2] != " ":
        return board[0][2]
    return None

def is_full(board):
    return all(board[r][c] != " " for r in range(3) for c in range(3))

def get_empty_cells(board):
    return [(r, c) for r in range(3) for c in range(3) if board[r][c] == " "]

def alpha_beta(board, depth, alpha, beta, is_maximizing):
    winner = check_winner(board)
    if winner == "X":
        return 10 - depth
    if winner == "O":
        return depth - 10
    if is_full(board):
        return 0

    if is_maximizing:
        max_eval = -math.inf
        for r, c in get_empty_cells(board):
            board[r][c] = "X"
            eval = alpha_beta(board, depth + 1, alpha, beta, False)
            board[r][c] = " "
            max_eval = max(max_eval, eval)
            alpha = max(alpha, eval)
            if beta <= alpha:
                break
        return max_eval
    else:
        min_eval = math.inf
        for r, c in get_empty_cells(board):
            board[r][c] = "O"
            eval = alpha_beta(board, depth + 1, alpha, beta, True)
            board[r][c] = " "
            min_eval = min(min_eval, eval)
            beta = min(beta, eval)
            if beta <= alpha:
                break
        return min_eval

def best_move(board):
    best_val = -math.inf
    move = None
    for r, c in get_empty_cells(board):
        board[r][c] = "X"
        move_val = alpha_beta(board, 0, -math.inf, math.inf, False)
        board[r][c] = " "
        if move_val > best_val:
            best_val = move_val
            move = (r, c)
    return move

def play_game():
    board = [[" "] * 3 for _ in range(3)]
    print("Tic-Tac-Toe: You = O, AI = X\n")
```

```
    print_board(board)

    while True:
        print("Your move (row col): ", end="")
        r, c = map(int, input().split())
        if board[r][c] != " ":
            print("Cell already taken!\n")
            continue
        board[r][c] = "O"
        print_board(board)

        if check_winner(board) == "O":
            print("You win!")
            break
        if is_full(board):
            print("It's a draw!")
            break

        print("AI is thinking...\n")
        r, c = best_move(board)
        board[r][c] = "X"
        print_board(board)

        if check_winner(board) == "X":
            print("AI wins!")
            break
        if is_full(board):
            print("It's a draw!")
            break

if __name__ == "__main__":
    play_game()
```

6. Implement a Sudoku solver using backtracking and constraint propagation techniques

```
def print_board(board):
    for i, row in enumerate(board):
        if i % 3 == 0 and i != 0:
            print("-" * 21)
        for j, val in enumerate(row):
            if j % 3 == 0 and j != 0:
                print("|", end=" ")
            print(val if val != 0 else ".", end=" ")
        print()
    print()

def is_valid(board, row, col, num):
    if num in board[row]:
        return False
    if num in [board[r][col] for r in range(9)]:
        return False
    box_r, box_c = (row // 3) * 3, (col // 3) * 3
    for r in range(box_r, box_r + 3):
        for c in range(box_c, box_c + 3):
            if board[r][c] == num:
                return False
    return True

def get_possible_values(board, row, col):
    if board[row][col] != 0:
        return set()
    possible = set(range(1, 10))
    possible -= set(board[row])
    possible -= set(board[r][col] for r in range(9))
    box_r, box_c = (row // 3) * 3, (col // 3) * 3
    for r in range(box_r, box_r + 3):
        for c in range(box_c, box_c + 3):
            possible.discard(board[r][c])
    return possible

def propagate_constraints(board):
    changed = True
    while changed:
        changed = False
        for row in range(9):
            for col in range(9):
                if board[row][col] == 0:
                    possible = get_possible_values(board, row, col)
                    if len(possible) == 0:
                        return False
                    if len(possible) == 1:
                        board[row][col] = possible.pop()
                        changed = True
    return True

def find_empty(board):
    min_possible = 10
    best_cell = None
    for row in range(9):
        for col in range(9):
            if board[row][col] == 0:
                possible = get_possible_values(board, row, col)
                if len(possible) < min_possible:
                    min_possible = len(possible)
                    best_cell = (row, col)
    return best_cell

def solve(board):
    if not propagate_constraints(board):
        return False

    cell = find_empty(board)
    if cell is None:
        return True

    row, col = cell
    possible = get_possible_values(board, row, col)

    for num in possible:
        backup = [r[:] for r in board]
```

```
            board[row][col] = num
            if solve(board):
                return True
            for r in range(9):
                board[r] = backup[r]

    return False

if __name__ == "__main__":
    board = [
        [5, 3, 0, 0, 7, 0, 0, 0, 0],
        [6, 0, 0, 1, 9, 5, 0, 0, 0],
        [0, 9, 8, 0, 0, 0, 0, 6, 0],
        [8, 0, 0, 0, 6, 0, 0, 0, 3],
        [4, 0, 0, 8, 0, 3, 0, 0, 1],
        [7, 0, 0, 0, 2, 0, 0, 0, 6],
        [0, 6, 0, 0, 0, 0, 2, 8, 0],
        [0, 0, 0, 4, 1, 9, 0, 0, 5],
        [0, 0, 0, 0, 8, 0, 0, 7, 9]
    ]

    print("Unsolved Board:")
    print_board(board)

    if solve(board):
        print("Solved Board:")
        print_board(board)
    else:
        print("No solution exists.")
```

7. Implement Backtracking search to solve the 8-Queens constraint satisfaction problem.

```
def print_board(queens, n=8):
    for row in range(n):
        line = ""
        for col in range(n):
            if queens[row] == col:
                line += "Q "
            else:
                line += ". "
        print(line)
    print()

def is_safe(queens, row, col):
    for r in range(row):
        c = queens[r]
        if c == col:
            return False
        if abs(r - row) == abs(c - col):
            return False
    return True

def backtrack(queens, row, n, solutions):
    if row == n:
        solutions.append(queens[:])
        return
    for col in range(n):
        if is_safe(queens, row, col):
            queens[row] = col
            backtrack(queens, row + 1, n, solutions)
            queens[row] = -1

def solve_n_queens(n=8):
    queens = [-1] * n
    solutions = []
    backtrack(queens, 0, n, solutions)
    return solutions

if __name__ == "__main__":
    n = 8
    solutions = solve_n_queens(n)

    print(f"Total solutions found: {len(solutions)}\n")

    print("Solution 1:")
    print_board(solutions[0], n)

    print("Solution 2:")
    print_board(solutions[1], n)

    print(f"Queen positions for Solution 1 (row: col):")
    for row, col in enumerate(solutions[0]):
        print(f"  Row {row + 1}: Column {col + 1}")
```

8. Develop a simple chatbot using predefined rules and pattern matching

```
import re
import random

responses = {
    r"hello|hi|hey|howdy": [
        "Hello! How can I help you?",
        "Hey there! What's up?",
        "Hi! Nice to meet you!"
    ],
    r"how are you|how do you do|how are things": [
        "I'm doing great, thanks for asking!",
        "I'm just a bot, but I'm feeling awesome!",
        "All good on my end! How about you?"
    ],
    r"what is your name|who are you|what are you": [
        "I'm RuleBot, a simple rule-based chatbot!",
        "You can call me RuleBot!",
        "I'm RuleBot, here to chat with you!"
    ],
    r"what can you do|help|what do you know": [
        "I can chat with you, answer basic questions, tell jokes, and more!",
        "I can talk about time, jokes, greetings, and general stuff!",
        "Try asking me a joke, the time, or just say hello!"
    ],
    r"tell me a joke|joke|make me laugh|funny": [
        "Why don't scientists trust atoms? Because they make up everything!",
        "Why did the computer go to the doctor? Because it had a virus!",
        "What do you call a fake noodle? An impasta!",
        "Why did the programmer quit? Because he didn't get arrays!"
    ],
    r"what time is it|current time|what's the time": [
        "I don't have a clock, but your device does — check the corner of your screen!"
    ],
    r"what is ai|tell me about ai|explain ai|artificial intelligence": [
        "AI stands for Artificial Intelligence — machines that simulate human thinking!",
        "AI is the ability of computers to perform tasks that normally require human intelligence.",
        "Artificial Intelligence includes things like machine learning, NLP, and robotics!"
    ],
    r"what is machine learning|explain machine learning|ml": [
        "Machine Learning is a subset of AI where machines learn from data!",
        "ML allows computers to improve automatically through experience without being explicitly programmed."
    ],
    r"who created you|who made you|who built you": [
        "I was created using Python and simple pattern matching rules!",
        "A developer built me using rule-based logic in Python!"
    ],
    r"bye|goodbye|see you|exit|quit": [
        "Goodbye! Have a great day!",
        "See you later! Take care!",
        "Bye! It was nice chatting with you!"
    ],
    r"thank you|thanks|thx": [
        "You're welcome!",
        "Happy to help!",
        "Anytime!    "
    ],
    r"good morning": [
        "Good morning! Hope you have a wonderful day!",
        "Morning! Rise and shine!"
    ],
    r"good night": [
        "Good night! Sweet dreams!",
        "Rest well! See you tomorrow!"
    ],
    r"what is python|tell me about python": [
        "Python is a high-level, easy-to-read programming language widely used in AI and web development!",
        "Python is one of the most popular languages, known for its simplicity and versatility!"
    ],
    r"favorite color|your color": [
        "I like all colors, but I think blue is quite calming!",
        "As a bot, I see everything in binary, but I'd say green!"
    ],
}

default_responses = [
    "I'm not sure I understand. Can you rephrase that?",
```

```
    "Hmm, I don't know about that. Try asking something else!",
    "That's interesting! But I don't have an answer for that.",
    "I'm still learning! Could you ask me something else?"
]

def get_response(user_input):
    user_input = user_input.lower().strip()
    for pattern, replies in responses.items():
        if re.search(pattern, user_input):
            return random.choice(replies)
    return random.choice(default_responses)

def chat():
    print("=" * 40)
    print("   Welcome to RuleBot!")
    print("   Type 'bye' to exit.")
    print("=" * 40)
    print()
    while True:
        user_input = input("You: ").strip()
        if not user_input:
            continue
        response = get_response(user_input)
        print(f"RuleBot: {response}\n")
        if re.search(r"bye|goodbye|see you|exit|quit", user_input.lower()):
            break

if __name__ == "__main__":
    chat()
```

9. Design an intelligent agent that guesses a number using feedback (higher/lower)

```
import random

def get_next_guess(low, high):
    return (low + high) // 2

def agent_guess(low, high, target):
    steps = []
    while low <= high:
        guess = get_next_guess(low, high)
        if guess == target:
            steps.append((guess, "Correct!"))
            return guess, steps
        elif guess < target:
            steps.append((guess, "Too Low"))
            low = guess + 1
        else:
            steps.append((guess, "Too High"))
            high = guess - 1
    return None, steps

def play_agent_vs_computer(low=1, high=100):
    target = random.randint(low, high)
    print(f"\nA number has been secretly chosen between {low} and {high}.")
    print("The intelligent agent will now guess it...\n")

    final_guess, steps = agent_guess(low, high, target)

    for i, (guess, feedback) in enumerate(steps, 1):
        print(f"Step {i}: Agent guesses {guess} → {feedback}")

    print(f"\nAgent found the number {target} in {len(steps)} steps!")

def play_human_mode(low=1, high=100):
    print(f"\nThink of a number between {low} and {high}.")
    print("The agent will guess it. Give feedback: 'h' = too high, 'l' = too low, 'c' = correct\n")

    steps = 0
    while low <= high:
        guess = get_next_guess(low, high)
        steps += 1
        print(f"Step {steps}: Agent guesses: {guess}")
        feedback = input("Your feedback (h/l/c): ").strip().lower()

        if feedback == "c":
            print(f"\nAgent guessed your number {guess} in {steps} steps!")
            return
        elif feedback == "l":
            low = guess + 1
        elif feedback == "h":
            high = guess - 1
        else:
            print("Invalid input. Enter h, l, or c.")
            steps -= 1

    print("Something went wrong. Did you change your number?")

def main():
    print("=" * 40)
    print("   Intelligent Number Guessing Agent")
    print("=" * 40)
    print("\nSelect Mode:")
    print("1. Agent guesses a random number automatically")
    print("2. Agent guesses YOUR number (you give feedback)")

    choice = input("\nEnter choice (1 or 2): ").strip()

    if choice == "1":
        play_agent_vs_computer()
    elif choice == "2":
        play_human_mode()
    else:
        print("Invalid choice. Please run again and enter 1 or 2.")

if __name__ == "__main__":
    main()
```

10. Implement Goal Stack Planning for solving the Blocks World problem and achieve a specified goal configuration.

```
def print_state(state):
    print("Current State:")
    for key, val in state.items():
        print(f"  {key}: {val}")
    print()

def on(block, location, state):
    return state.get(block) == location

def clear(block, state):
    return block not in state.values()

def on_table(block, state):
    return state.get(block) == "table"

def move(block, from_loc, to_loc, state):
    print(f"  ACTION: Move {block} from {from_loc} to {to_loc}")
    state[block] = to_loc
    print_state(state)

def achieve(goal, state, stack):
    if check_goal(goal, state):
        return
    if goal[0] == "on":
        _, block, location = goal
        stack.append(goal)
        stack.append(("clear", block))
        stack.append(("clear", location))
    elif goal[0] == "clear":
        _, block = goal
        blocker = find_blocker(block, state)
        if blocker:
            stack.append(goal)
            stack.append(("on", blocker, "table"))
    elif goal[0] == "on_table":
        _, block = goal
        stack.append(goal)
        stack.append(("clear", block))

def find_blocker(block, state):
    for b, loc in state.items():
        if loc == block:
            return b
    return None

def check_goal(goal, state):
    if goal[0] == "on":
        return on(goal[1], goal[2], state)
    elif goal[0] == "clear":
        return clear(goal[1], state)
    elif goal[0] == "on_table":
        return on_table(goal[1], state)
    return False

def execute_action(goal, state):
    if goal[0] == "on":
        _, block, location = goal
        from_loc = state.get(block)
        move(block, from_loc, location, state)
    elif goal[0] == "on_table":
        _, block = goal
        from_loc = state.get(block)
        move(block, from_loc, "table", state)

def goal_stack_planning(initial_state, goals):
    state = dict(initial_state)
    stack = list(goals[::-1])

    print("=" * 40)
    print("Initial State:")
    print_state(state)
    print("Goals:", goals)
    print("=" * 40)
    print()
```

```
    iterations = 0
    max_iterations = 100

    while stack and iterations < max_iterations:
        iterations += 1
        current = stack[-1]

        if check_goal(current, state):
            stack.pop()
        elif current[0] in ("on", "clear", "on_table"):
            blocker = None
            if current[0] == "on":
                _, block, location = current
                if not clear(block, state):
                    blocker = find_blocker(block, state)
                    if blocker:
                        stack.append(("on", blocker, "table"))
                        stack.append(("clear", blocker))
                        continue
                if not clear(location, state) and location != "table":
                    blocker = find_blocker(location, state)
                    if blocker:
                        stack.append(("on", blocker, "table"))
                        stack.append(("clear", blocker))
                        continue
                execute_action(current, state)
                stack.pop()

            elif current[0] == "clear":
                _, block = current
                blocker = find_blocker(block, state)
                if blocker:
                    stack.append(("on", blocker, "table"))
                    stack.append(("clear", blocker))
                else:
                    stack.pop()

            elif current[0] == "on_table":
                _, block = current
                if not clear(block, state):
                    blocker = find_blocker(block, state)
                    if blocker:
                        stack.append(("on", blocker, "table"))
                        stack.append(("clear", blocker))
                        continue
                execute_action(current, state)
                stack.pop()
        else:
            stack.pop()

    print("=" * 40)
    print("Final State:")
    print_state(state)
    print("Goal achieved!" if iterations < max_iterations else "Could not complete.")
    print("=" * 40)

if __name__ == "__main__":
    initial_state = {
        "A": "table",
        "B": "table",
        "C": "A"
    }

    goals = [
        ("on", "A", "B"),
        ("on", "B", "table"),
        ("on", "C", "A")
    ]

    goal_stack_planning(initial_state, goals)
```

11. Develop a game-playing agent using the Minimax algorithm for optimal decision-making

```
import math

def print_board(board):
    for i, row in enumerate(board):
        print(" | ".join(row))
        if i < 2:
            print("-" * 9)
    print()

def check_winner(board):
    for row in board:
        if row[0] == row[1] == row[2] and row[0] != " ":
            return row[0]
    for col in range(3):
        if board[0][col] == board[1][col] == board[2][col] and board[0][col] != " ":
            return board[0][col]
    if board[0][0] == board[1][1] == board[2][2] and board[0][0] != " ":
        return board[0][0]
    if board[0][2] == board[1][1] == board[2][0] and board[0][2] != " ":
        return board[0][2]
    return None

def is_full(board):
    return all(board[r][c] != " " for r in range(3) for c in range(3))

def get_empty_cells(board):
    return [(r, c) for r in range(3) for c in range(3) if board[r][c] == " "]

def minimax(board, depth, is_maximizing):
    winner = check_winner(board)
    if winner == "X":
        return 10 - depth
    if winner == "O":
        return depth - 10
    if is_full(board):
        return 0

    if is_maximizing:
        best = -math.inf
        for r, c in get_empty_cells(board):
            board[r][c] = "X"
            score = minimax(board, depth + 1, False)
            board[r][c] = " "
            best = max(best, score)
        return best
    else:
        best = math.inf
        for r, c in get_empty_cells(board):
            board[r][c] = "O"
            score = minimax(board, depth + 1, True)
            board[r][c] = " "
            best = min(best, score)
        return best

def best_move(board):
    best_val = -math.inf
    move = None
    for r, c in get_empty_cells(board):
        board[r][c] = "X"
        move_val = minimax(board, 0, False)
        board[r][c] = " "
        if move_val > best_val:
            best_val = move_val
            move = (r, c)
    return move

def get_human_move(board):
    while True:
        try:
            r, c = map(int, input("Enter row and col (0-2): ").split())
            if board[r][c] == " ":
                return r, c
            else:
                print("Cell already taken. Try again.")
```

```
        except:
            print("Invalid input. Enter two numbers like: 1 2")

def play_game():
    board = [[" "] * 3 for _ in range(3)]

    print("=" * 30)
    print("   Tic-Tac-Toe: Minimax Agent")
    print("   You = O  |  AI = X")
    print("=" * 30)
    print("\nBoard positions:")
    print("0,0 | 0,1 | 0,2")
    print("1,0 | 1,1 | 1,2")
    print("2,0 | 2,1 | 2,2\n")

    print("Who goes first?")
    print("1. You (O)")
    print("2. AI (X)")
    choice = input("Enter choice (1 or 2): ").strip()

    human_first = choice != "2"
    current_turn = "human" if human_first else "ai"

    print()
    print_board(board)

    while True:
        if current_turn == "human":
            print("Your turn (O):")
            r, c = get_human_move(board)
            board[r][c] = "O"
            print()
            print_board(board)
            winner = check_winner(board)
            if winner == "O":
                print("Congratulations! You win!")
                break
            if is_full(board):
                print("It's a draw!")
                break
            current_turn = "ai"

        else:
            print("AI is thinking...\n")
            r, c = best_move(board)
            board[r][c] = "X"
            print(f"AI played at row {r}, col {c}\n")
            print_board(board)
            winner = check_winner(board)
            if winner == "X":
                print("AI wins! Better luck next time.")
                break
            if is_full(board):
                print("It's a draw!")
                break
            current_turn = "human"

if __name__ == "__main__":
    play_game()
```